SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE CODE: ITA 0603 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 10%

QUESTIONS: 15 **Total Marks:25**

Annexure A

DEBUGGING & OPTIMIZATION TASK QUESTIONS

Code of Conduct:

I Peta Harsha Vardhan (192311039) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Short Answer Test

1. A perceptron model is trained on a dataset that is known to be linearly separable, yet the algorithm fails to converge even after many iterations. Analyze the possible causes of this issue, including improper learning rate selection, incorrect weight initialization, bias handling errors, or flawed implementation of the update rule. Propose debugging strategies to identify the root cause and suggest optimization techniques to ensure faster and stable convergence. (5 Marks)

2. While training a multilayer perceptron using the backpropagation algorithm, the network exhibits slow convergence and gets stuck in local minima. Examine the potential reasons such as vanishing gradients, inappropriate activation functions, poor weight initialization, or suboptimal learning rates. Describe step-by-step debugging approaches to trace the issue and recommend optimization techniques like adaptive learning rates, normalization, or advanced optimizers to improve performance. (5 Marks)

3. A genetic algorithm designed for hypothesis space search is producing inconsistent and suboptimal solutions across multiple runs. Investigate possible issues such as improper encoding of chromosomes, incorrect fitness function design, premature convergence, or lack of diversity in the population. Explain how to debug these problems systematically and suggest optimization strategies like mutation tuning, selection pressure adjustment, and elitism to enhance solution quality and convergence speed. (5 Marks)

4. A deep neural network combined with genetic programming is used for solving a complex prediction problem, but it suffers from overfitting and high computational cost. Identify debugging steps to analyze issues related to model complexity, over-parameterization, and insufficient training data. Propose optimization techniques such as regularization, dropout, pruning, and efficient evaluation models to balance accuracy and computational efficiency while improving generalization capability. (5 Marks)

5. A decision tree model trained on a large dataset shows high accuracy on training data but poor performance on unseen data. Analyze possible causes such as overfitting, deep tree structure, or noisy data. Propose debugging steps and optimization methods like pruning, cross-validation, and feature selection to improve generalization. (5 Marks)

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	30	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	10	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1

Concept Involved

Perceptron learning rule, convergence on linearly separable data, learning rate, weight initialisation, bias handling, debugging and optimisation of training.

Definition

The perceptron updates its weights only when it misclassifies an example: $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(t - y)\,\mathbf{x}$ and $\mathbf{b} \leftarrow \mathbf{b} + \eta\,(t - y)$. The **Perceptron Convergence Theorem** guarantees that if the data are linearly separable, this rule finds a separating line after a finite number of updates. Therefore, failure to converge on separable data is a symptom of a bug, a data problem or a wrong setting, and not a limitation of the model.

Problem Scenario / Given Data

Item	Observation
Dataset	Known to be linearly separable
Training	Many iterations (epochs) completed
Symptom	Number of misclassified samples per epoch never reaches 0
Expected	Zero errors after a finite number of updates

Answer (Step-by-Step Solution)

Step 1: Analyse the possible causes

Possible cause	What goes wrong	How to check
Bias handling error	Without a bias the decision line must pass through the origin, so many separable data sets cannot be separated; the bias may be missing or not updated	Check that b exists, is added to net, and is updated with the same error term
Flawed update rule	Wrong sign (y – t instead of t – y), missing the factor x, updating when the answer is correct, or never updating the bias	Hand-compute one update and compare with the code
Improper learning rate	For the pure perceptron with zero start, η only scales the weights; but η = 0 or negative gives no learning, and with a delta-rule / sigmoid unit a large η makes the weights oscillate	Try η = 0.01, 0.1, 1 and compare the error curve
Incorrect weight initialisation	Very large or NaN starting weights need many corrections or never recover	Print the initial weights; use zeros or small random values
Label / threshold mismatch	Labels coded –1/+1 while the output is 0/1, or step uses > instead of ≥	Print labels and predictions side by side
Data problems	Unscaled features (one feature 1000 times larger), or label noise / duplicate points with different labels, so data are not truly separable	Standardise features; run a separability test

Step 2: Debugging strategy (step by step)

1. **Confirm separability** with a 2-D plot, a hard-margin linear SVM or a linear-programming check.
2. **Test on a tiny problem** with a known answer, for example the AND gate (below).

3. **Log every epoch:** number of errors, weights and bias; a flat error count means no learning.
4. **Hand-check one update** against the code, using the update rule above.
5. **Plug in a known separating line.** If the code still reports errors, the bug is in the prediction or evaluation step.
6. **Check labels, threshold, array shapes and shuffling;** fix the random seed so that the run can be repeated.
7. **Change one setting at a time** (η, initialisation, scaling) and observe the error curve.

Step 3: Worked test – AND gate (η = 1, zero start, y = 1 if net ≥ 0)

Epoch	Errors with bias	Weights w and bias b	Errors without bias	Weights w
1	2	w = (1, 1), b = 0	4	(0, 0)
2	3	w = (2, 1), b = –1	4	(0, 0)
3	3	w = (2, 1), b = –2	4	(0, 0)
4	2	w = (2, 2), b = –2	4	(0, 0)
5	1	w = (2, 1), b = –3	4	(0, 0)
6	**0**	**converged**	4	(0, 0)

With a bias the perceptron converges in 6 epochs to w = (2, 1), b = –3 (net = $2x_1 + x_2 - 3$, which is ≥ 0 only for (1, 1)). Without a bias, the input (0, 0) always gives net = 0, so it is always classified as 1 (wrong) and the weights just cycle back to (0, 0): 4 errors in every epoch, forever. **Root cause found: missing bias.**

Step 4: Corrected implementation

```
def train(X, t, eta=0.1, epochs=100):
    X = np.c_[X, np.ones(len(X))]    # bias trick: extra input 1
    w = np.zeros(X.shape[1])         # zero initial weights
    for ep in range(epochs):
        errors = 0
        for i in np.random.permutation(len(X)):  # shuffle
            y = 1 if X[i] @ w >= 0 else 0
            if y != t[i]:
                w += eta * (t[i] - y) * X[i]   # correct sign
                errors += 1
        print(ep, errors)
        if errors == 0: break            # all correct: stop
    return w
```

Step 5: Optimisation for faster and stable convergence

- **Standardise features** (z-score) so that no feature dominates and the updates are balanced.
- **Use the bias trick and shuffle** the data every epoch to avoid cycling.
- **Choose a suitable η** (0.01–1) or decay it slowly; start from zero or small random weights.
- **Stop when errors = 0** and set a maximum number of epochs as a safeguard.
- **Pocket or averaged perceptron** keeps the best (or average) weights, which stays stable if a few labels are noisy.

- If the data are not truly separable, switch to a **margin-based or smooth model** (logistic regression, SVM, MLP).

Application
The same checks (tiny test case, per-epoch logging, bias and sign verification) are used when debugging any gradient-based learner, and the perceptron is the standard building block used to teach neural-network training.

Conclusion
A perceptron must converge on separable data, so non-convergence points to a bias error, a wrong update rule, a data or label problem, or bad settings. The AND-gate test exposed the missing bias (4 errors every epoch without it, 0 errors after 6 epochs with it). Adding the bias, using the correct update, scaling the inputs, shuffling and logging errors gives fast and stable convergence.

Question 2

Concept Involved
Backpropagation in a multilayer perceptron, vanishing gradients, activation functions, weight initialisation, learning rate, gradient checking, adaptive optimisers and normalisation.

Definition
Backpropagation computes the gradient of the loss with respect to every weight using the chain rule and updates $w \leftarrow w - \eta\, \partial E/\partial w$. Training is **slow** when these gradients are tiny (vanishing), badly scaled or noisy, and it appears **stuck** when the weights reach a plateau, a saddle point or a poor minimum. In deep MLPs, truly bad local minima are rare; plateaus and saddle points caused by saturated units are the more common reason.

Problem Scenario / Given Data

Item	Observation
Model	Multilayer perceptron trained with backpropagation
Symptom 1	Loss falls very slowly
Symptom 2	Loss stops at a high value (appears stuck in a local minimum)

Answer (Step-by-Step Solution)

Step 1: Analyse the possible causes

Possible cause	What goes wrong	How to check
Vanishing gradients	The sigmoid slope is at most 0.25, so the error signal shrinks at every layer (see the figure); early layers hardly learn	Print the gradient norm of each layer; it falls by orders of magnitude towards the input
Inappropriate activation / loss	Sigmoid or tanh saturate; sigmoid with squared error gives a tiny gradient when the output is confidently wrong	Plot activation histograms; values piled near 0 or 1 mean saturation
Poor weight initialisation	All-zero weights make every neuron in a layer identical (symmetry); very large weights saturate units	Check that the weights differ and that activations have a sensible spread
Suboptimal learning rate	Too small: very slow; too large: the loss oscillates or diverges	Learning-rate range test; watch the loss curve

Possible cause	What goes wrong	How to check
Unscaled inputs / data errors	Features with very different ranges make the loss surface narrow and elongated; wrong labels give no learnable signal	Check means and standard deviations of inputs; inspect labels
Gradient implementation bug	Wrong derivative or missing term in backpropagation	Gradient checking (below)

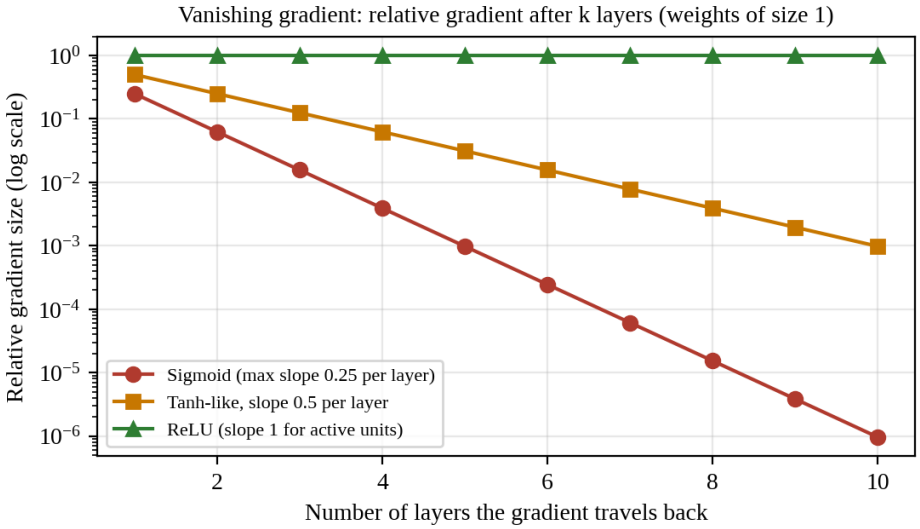


Fig. 1: Relative gradient size after k layers. Sigmoid: $0.25^{5} \approx 0.001$ after 5 layers and $0.25^{10} \approx 10^{-6}$ after 10 layers

Step 2: Step-by-step debugging approach

1. **Overfit a tiny batch** (10 samples). A correct network must reach almost zero loss; if not, there is a bug.
2. **Gradient checking:** compare the backpropagation gradient with the numerical one, $[J(\theta + \varepsilon) - J(\theta - \varepsilon)] / 2\varepsilon$ with $\varepsilon = 10^{-4}$. A relative error below 10^{-7} is good; above 10^{-3} shows a bug.
3. **Monitor per-layer gradient norms and activation histograms** to detect vanishing gradients and saturation.
4. **Learning-rate range test:** raise η from 10^{-6} to 1 over a few hundred steps and pick the value just before the loss rises.
5. **Check the data pipeline:** standardise inputs, verify labels, and confirm that the loss matches the task (cross-entropy for classification).
6. **Simplify:** reduce depth and compare with a linear baseline; add one component at a time until the problem returns.

Step 3: Optimisation techniques

Technique	How it works	Benefit
ReLU / Leaky ReLU	f(z) = max(0, z); slope 1 for active units	Gradients do not vanish; faster training
He / Xavier initialisation	Var(w) = 2 / n_in (He, for ReLU); Var(w) = 2 / (n_in + n_out) (Xavier)	Keeps activation and gradient sizes stable across layers
Input standardisation and batch normalisation	Scale to zero mean, unit variance (inputs and layer outputs)	Rounder loss surface; allows a larger η

Technique	How it works	Benefit
Momentum	$v \leftarrow \beta v - \eta \nabla J,\ w \leftarrow w + v\ (\beta \approx 0.9)$	Rolls through plateaus and shallow minima, reduces oscillation
Adaptive optimisers (RMSprop, Adam)	Per-weight step size from running averages of gradient and squared gradient (Adam: $\beta_1 = 0.9$, $\beta_2 = 0.999$)	Fast, robust convergence with little tuning
Learning-rate schedule	Decay η (step or cosine) during training	Fast start, fine convergence
Mini-batch SGD, residual links, gradient clipping	Noisy gradients; skip connections; limit gradient norm	Escapes poor regions; helps very deep nets; prevents explosion

Application
These checks and fixes are routine when training deep networks for image, speech and tabular prediction; most modern deep-learning libraries use ReLU, He initialisation, batch normalisation and Adam as defaults for this reason.

Conclusion
Slow convergence and apparent local minima in an MLP usually come from vanishing gradients, saturating activations, poor initialisation, unscaled inputs or a badly chosen learning rate. They are traced by overfitting a tiny batch, gradient checking, and monitoring gradient norms and activations. Switching to ReLU with He/Xavier initialisation, normalising inputs and layers, and training with momentum or Adam and a learning-rate schedule restores fast and reliable convergence.

Question 3

Concept Involved
Genetic Algorithm for hypothesis space search, chromosome encoding, fitness function, premature convergence, population diversity, mutation, selection pressure and elitism.

Definition
A genetic algorithm (GA) searches a hypothesis space by evolving a population through selection, crossover and mutation. Its outcome depends on four design choices: how a hypothesis is **encoded**, how its **fitness** is measured, how strongly **selection** favours the best, and how much **diversity** the operators maintain. **Premature convergence** occurs when the population becomes almost identical before the global optimum is found.

Problem Scenario / Given Data

Item	Observation
Task	Hypothesis space search with a GA
Symptom	Different runs return different and suboptimal solutions
Possible issues	Encoding, fitness design, premature convergence, lack of diversity

Answer (Step-by-Step Solution)

Step 1: Analyse the possible causes

Possible cause	What goes wrong	How to check
Improper chromosome encoding	Plain binary coding has "Hamming cliffs": 7 = 0111 and 8 = 1000 differ in 4 bits, so neighbouring values are far apart in gene space; also invalid or redundant chromosomes, or too few bits for the required precision	Decode random chromosomes and check validity; measure bit distance between neighbours
Incorrect fitness function	Does not reflect the real goal, wrong sign (minimising instead of maximising), negative values with roulette selection, flat regions, or noisy evaluation	Test on a function with a known optimum; plot fitness against the parameter
Premature convergence	A few super-fit individuals dominate selection and fill the population with copies	Average fitness approaches best fitness very early
Lack of diversity	Small population, very low mutation rate, strong crossover of similar parents	Count unique chromosomes; mean pairwise Hamming distance near 0
Randomness / operator bugs	Results vary because of different random seeds; wrong crossover point or mutation applied to a copy that is not stored	Fix the seed; unit-test each operator

Step 2: Systematic debugging

1. **Benchmark first:** run the GA on a function with a known answer, for example $f(x) = x^2$ on 0–31 (optimum $x = 31$). It should find it in nearly every run.
2. **Log per generation:** best, average and worst fitness, and a diversity measure (unique chromosomes, mean pairwise Hamming distance).
3. **Read the curves:** best fitness flat early with average catching up means premature convergence; average and best both low and noisy means a weak fitness function or too much mutation.
4. **Unit-test each operator** (selection frequencies, crossover keeps gene count, mutation flips about $L \times p_m$ bits).
5. **Run 30 independent seeds** and report the mean ± standard deviation of the best fitness, instead of judging from a single run.
6. **Change one parameter at a time** (population size, mutation rate, selection) and compare the statistics.

Step 3: Numerical illustrations

Selection pressure. Fitness values {1000, 10, 10, 10, 10}: roulette gives the best individual 1000 / 1040 = 96% of the selections, so the next generation is nearly all copies of it. Rank selection (ranks 5, 4, 3, 2, 1) gives 33.3%, 26.7%, 20%, 13.3% and 6.7%, which keeps weaker but possibly useful individuals alive.

Encoding. Gray code, $g = n \text{ XOR } (n >> 1)$: 7 → 0100 and 8 → 1100 differ in only 1 bit, so a one-bit mutation can move between neighbouring values.

Mutation rate. For $L = 10$ genes, $p_m \approx 1 / L = 0.1$ per gene gives about one flipped gene per child; 0.001 would almost never change anything.

Step 4: Optimisation strategies

Strategy	Action	Effect
Better encoding / fitness	Use Gray or real-valued coding; scale fitness; add penalties for invalid hypotheses	Smoother search landscape; correct goal
Mutation tuning	$p_m \approx 1/L$, or adaptive (higher when diversity is low, lower later)	Restores diversity and fine-tunes late in the run
Selection pressure	Tournament selection (k = 2–5), rank selection or fitness scaling	Controls how fast the population converges
Elitism	Copy the best 1–2 individuals unchanged into the next generation	Best solution is never lost; faster, steadier convergence
Population and crossover	Larger population (50–200); two-point or uniform crossover; crossover rate 0.6–0.9	More exploration of the hypothesis space
Diversity maintenance	Fitness sharing / niching, random immigrants, restart on stagnation	Avoids local optima

```
gray = n ^ (n >> 1)                          # Gray encoding
def tournament(pop, fit, k=3):
    idx = random.sample(range(len(pop)), k)
    return pop[max(idx, key=lambda i: fit[i])]
elite = sorted(zip(fit, pop), reverse=True)[:2]  # elitism
```

Application

A tuned GA is used for feature selection, hyper-parameter and architecture search, scheduling and design optimisation, where reliable and repeatable results across runs matter.

Conclusion

Inconsistent and poor GA results come from weak encoding, a badly designed fitness function, excessive selection pressure and loss of diversity. They are found systematically by benchmarking on a known function, logging best, average and diversity curves, testing the operators and repeating runs over many seeds. Gray or real coding, a correct scaled fitness, tuned mutation (about 1/L), tournament or rank selection and elitism give better, faster and more repeatable solutions.

Question 4

Concept Involved

Deep neural network combined with genetic programming, overfitting, over-parameterisation, regularisation, dropout, pruning, and computational efficiency of fitness evaluation.

Definition

Genetic programming (GP) evolves programs or tree-structured expressions (here, assumed to evolve features or the network structure for a deep neural network, DNN). **Overfitting** occurs when the model memorises the training data and fails on new data. In such hybrids it is made worse by over-parameterisation, small data and GP **bloat** (trees growing without improving accuracy), while cost is high because every GP candidate needs a full DNN training run to compute its fitness.

Problem Scenario / Given Data

Item	Observation
Model	DNN combined with genetic programming for a complex prediction task
Symptom 1	Overfitting (training accuracy much higher than validation accuracy)
Symptom 2	High computational cost

Answer (Step-by-Step Solution)

Step 1: Debugging steps

Issue	Debugging step	Finding that confirms it
Model complexity / over-parameterisation	Count the parameters and compare with the number of samples; plot training and validation curves	Parameters far exceed samples; training error keeps falling while validation error rises
Insufficient training data	Plot a learning curve (accuracy against training-set size)	Validation accuracy still rising as data increase, with a large gap
Fitness overfitting in GP	Check that GP fitness is computed on a separate validation set, not on the training data	Best GP individual scores very high on training but low on validation
GP bloat	Plot average tree size against generation	Size keeps growing while fitness is flat
High computational cost	Profile time per fitness evaluation, memory use and number of evaluations	Total cost = population × generations × training time

Step 2: Numerical illustration

Over-parameterisation. A DNN with layers 20-128-64-1 has $(20 \times 128 + 128) + (128 \times 64 + 64) + (64 + 1) = 2{,}688 + 8{,}256 + 65 =$ **11,009 parameters**. With only 500 training samples this is about 22 parameters per sample, so the network can easily memorise the data.

Cost. Population 50 × 30 generations = 1,500 fitness evaluations. At 100 s of training each, the cost is 1,500 × 100 s = 150,000 s ≈ **41.7 hours**.

Step 3: Optimisation to improve generalisation

Technique	How it is applied	Effect
L2 regularisation (weight decay)	Loss = $E + \lambda \Sigma w^2$	Keeps weights small; smoother model
Dropout	Randomly switch off 20–50% of hidden neurons during training	Prevents co-adaptation; acts like an ensemble
Early stopping and data augmentation	Stop at the best validation loss; create extra samples	Less memorisation; more effective data
GP parsimony pressure	Fitness = accuracy – λ × tree size; limit the maximum depth	Controls bloat and prefers simple models
Pruning	Remove weights with the smallest magnitude (or entire neurons / sub-trees) and fine-tune	Smaller, faster model with almost the same accuracy

Technique	How it is applied	Effect
Efficient evaluation	Evaluate candidates on a data subset or for a few epochs, use a surrogate (predictive) fitness model, cache repeated individuals, evaluate in parallel	Large reduction in search time
Model compression and transfer learning	Quantisation, knowledge distillation, pre-trained layers	Lower cost and less data needed

Step 4: Effect of the optimisations (illustrative figures)

Parsimony: candidate A has accuracy 0.95 with 80 nodes, candidate B has 0.93 with 20 nodes; with $\lambda = 0.001$, fitness(A) = 0.95 – 0.08 = 0.87 and fitness(B) = 0.93 – 0.02 = 0.91, so the simpler B is selected and is likely to generalise better.

Pruning: removing 70% of the 11,009 weights leaves about 3,303 (11,009 × 0.3), which is roughly 6.6 parameters per sample instead of 22.

Cheaper evaluation: screening each candidate with a subset and few epochs (20 s instead of 100 s) reduces the search to 1,500 × 20 s = 30,000 s ≈ **8.3 hours**, an 80% saving; only the finalists are fully trained.

Application

Evolutionary design combined with deep networks is used in neuroevolution, automated feature construction, and neural architecture search, where regularisation and cheap fitness evaluation make the approach practical.

Conclusion

Overfitting in the DNN–GP hybrid is caused by too many parameters for too little data, fitness measured on training data and GP bloat, and the high cost comes from repeated full training of every candidate. Learning curves, parameter counting and profiling reveal the causes. Regularisation, dropout, early stopping, parsimony pressure and pruning improve generalisation, while subset or surrogate evaluation, caching and parallelism cut the computational cost.

Question 5

Concept Involved

Decision tree learning, overfitting, tree depth, noisy data, pruning (pre- and post-), cross-validation and feature selection.

Definition

A decision tree splits the data on features (using entropy / information gain or Gini impurity) until the leaves are pure. If it is grown without limit, it can fit every training example, including noise, giving 100% training accuracy but poor results on unseen data, which is **overfitting** (high variance). **Pruning** removes branches that do not improve generalisation.

Problem Scenario / Given Data

Item	Observation
Model	Decision tree trained on a large dataset
Training accuracy	High

Item	Observation
Accuracy on unseen data	Poor (large train–test gap)

Answer (Step-by-Step Solution)

Step 1: Analyse the possible causes

Possible cause	What goes wrong	How to check
Overfitting	The tree memorises training examples, including random variations	Large gap between training and validation accuracy
Deep tree structure	Very deep tree with many leaves, each covering only 1–2 samples	Check depth, number of leaves and samples per leaf
Noisy data / wrong labels	The tree creates special branches for mislabeled points	Inspect misclassified training points; look for conflicting duplicates
Irrelevant features / ID columns	Features such as an ID or timestamp split the data perfectly but mean nothing (data leakage)	Examine the feature-importance ranking
Unreliable evaluation	One lucky or unlucky split hides the real error	Compare scores over k-fold cross-validation

Step 2: Debugging steps

1. **Compare training and validation accuracy.** Illustration: unpruned tree 100% training, 72% test, a gap of 28 percentage points.
2. **Inspect the tree:** depth, number of leaves, samples per leaf and leaf purity.
3. **Plot accuracy against tree depth** (Fig. 2). Training accuracy keeps rising, while validation accuracy peaks and then falls.
4. **Check the data:** noisy or mislabeled records, duplicates, identifier columns and leakage.
5. **Examine feature importance** and remove features that look suspicious or irrelevant.
6. **Use k-fold cross-validation** to get a stable estimate and its variance.

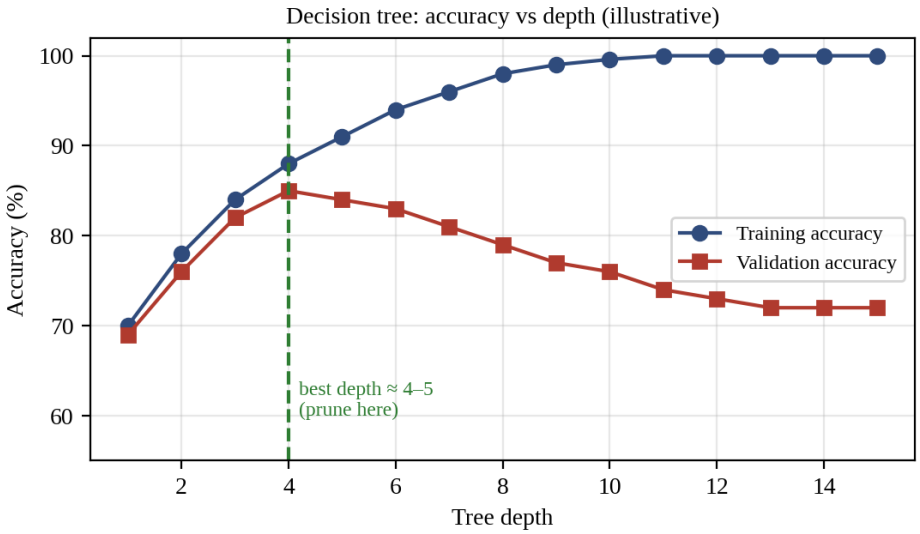


Fig. 2: Training and validation accuracy against tree depth (illustrative)

Step 3: Optimisation methods

Method	How it works	Effect
Pre-pruning	Limit max_depth, min_samples_split, min_samples_leaf or require a minimum information gain	Stops growth before noise is learned
Post-pruning	Grow the full tree, then remove branches by cost-complexity pruning, $R_\alpha(T) = R(T) + \alpha \times$ (number of leaves), or reduced-error pruning on validation data	Simpler tree with better generalisation
Cross-validation	Choose depth, leaf size and α by k-fold cross-validation (grid search)	Reliable hyper-parameters; avoids tuning to one split
Feature selection	Keep features with high information gain or importance; drop IDs and noisy features	Fewer misleading splits; faster model
Data cleaning / more data	Correct or remove noisy labels; collect more samples	Less noise to memorise
Ensembles	Random forest or gradient boosting (many trees, averaged)	Much lower variance

Step 4: Numerical illustration of cost-complexity pruning (α = 0.005)

Tree	Training error R(T)	Leaves	$R_\alpha(T) = R + \alpha \times$ leaves
Full tree	0.05	40	$0.05 + 0.005 \times 40 = 0.25$
Pruned tree	0.12	8	$0.12 + 0.005 \times 8 = 0.16$ (chosen)

The pruned tree has the smaller penalised cost, so it is preferred. After pruning to depth 5, the illustrative accuracy becomes 88% training and 84% test, so the gap falls from 28 to 4 points.

Step 5: Implementation with cross-validated pruning

```
params = {"max_depth": [3, 4, 5, 6, 8],
          "min_samples_leaf": [5, 10, 20],
          "ccp_alpha": [0.0, 0.001, 0.005, 0.01]}
tree = DecisionTreeClassifier(random_state=0)
grid = GridSearchCV(tree, params, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)   # best by 5-fold CV
print(grid.score(X_test, y_test))            # unseen data
```

Application

Pruned and cross-validated trees are used in credit scoring, medical diagnosis and customer analytics, where the rules must be understandable and the model must perform on future data.

Conclusion

High training accuracy with poor test accuracy shows that the tree is overfitting, usually because it is too deep or is learning noise or irrelevant features. The gap, the depth–accuracy curve, leaf sizes and feature importance reveal the problem. Pre- and post-pruning, cross-validated hyper-parameter selection, feature selection and data cleaning (or an ensemble such as a random forest) reduce the gap and improve generalisation.